		Nilai	Paraf Dosen
UINSSC UNIVERSITAS ISLAM NEGERI SIBER SYEKH NURJATI CIREBON	**LEMBAR KERJA MANDIRI 1** **COMPUTER VISION** T.A. Genap 2025/2026		

A. **Identitas**

Topik : Pra Pemrosesan dan Pembersihan Data
NIM : 2388010004
Nama : Abdul Kharis
Kelas : INF 6A

B. Tujuan

1. Mahasiswa memahami konsep Pra Pemrosesan dalam Computer Vision.
2. Mahasiswa mampu melakukan Pembersihan Data.
3. Mahasiswa mampu mengimplementasikan Basic Image Processing dalam data citra

C. Instruksi Pengerjaan

1. Tahap 1 – Pertemuan ke 2

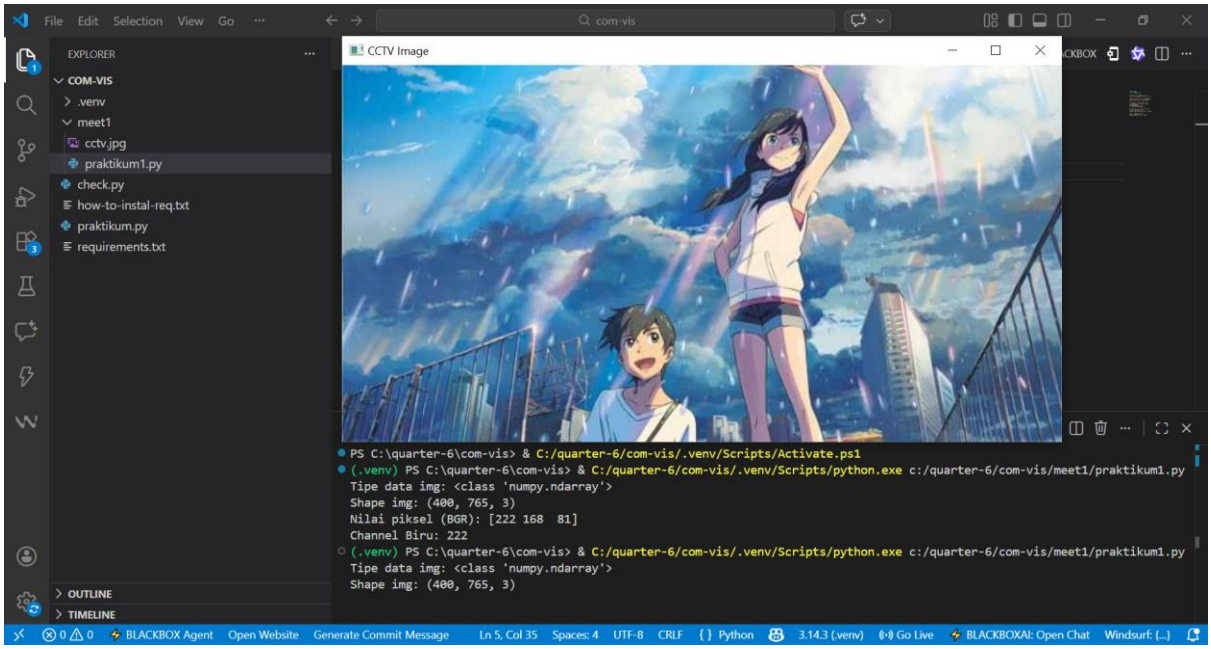
1. Mengapa nilai piksel terdiri dari 3 channel?

   Nilai piksel terdiri dari tiga channel karena gambar berwarna disusun dari tiga warna dasar, yaitu biru, hijau, dan merah.
   Setiap channel menyimpan nilai intensitas warna yang berbeda sehingga satu piksel dapat menampilkan warna tertentu.
   Jika gambar hanya memiliki satu channel, maka gambar tersebut hanya akan menampilkan warna hitam dan putih.

2. Apa hubungan nilai piksel dan warna?

   Nilai piksel menentukan warna yang terlihat pada gambar.
   Nilai pada setiap channel menunjukkan seberapa kuat warna tersebut ditampilkan.
   Perbedaan kombinasi nilai pada ketiga channel akan menghasilkan warna yang berbeda pada setiap piksel.

3. Capture



Screenshot sintak kode program dibawah ini

2. Tahap 2 – Pertemuan ke 3

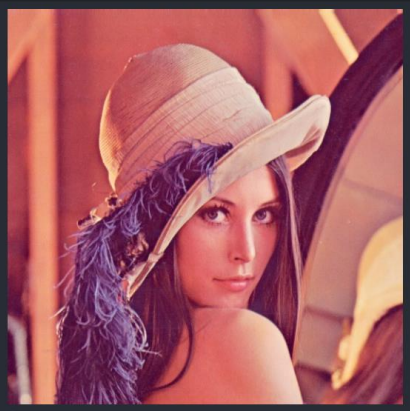
```
!pip install opencv-python
!pip install matplotlib
```

Python

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.
```

Melakukan pengecekan instalasi library OpenCV dan Matplotlib pada lingkungan Python 3.12, dengan hasil sistem menyatakan status "Requirement already satisfied" yang berarti library sudah terpasang dan siap digunakan untuk pengolahan citra.

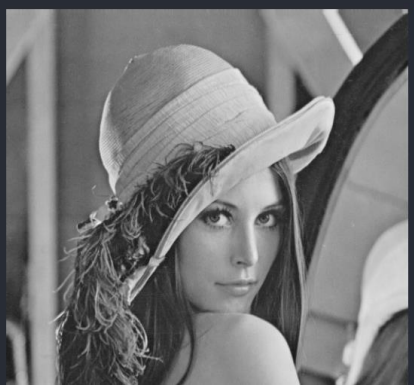
```python
import os
import numpy as np
import matplotlib.pyplot as plt
import cv2 # Correct
from google.colab.patches import cv2_imshow

# Membaca Citra
img = cv2.imread('gambar.jpg')

#menampilkan citra
cv2_imshow(img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Melakukan pemanggilan library OpenCV dan Matplotlib untuk membaca file citra digital (gambar.jpg) ke dalam variabel, kemudian menampilkannya secara visual pada notebook menggunakan fungsi cv2_imshow.

```python
image = cv2.imread('gambar.jpg')

# Koversi Citra ke Grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Menampilkan Citra Grayscale
print('Citra GrayScale')
cv2_imshow(gray_image)
cv2.waitKey(0)
cv2.destroyAllWindows()

# Menyimpan Citra dalam format PNG
cv2.imwrite('gambar.png', gray_image)
```

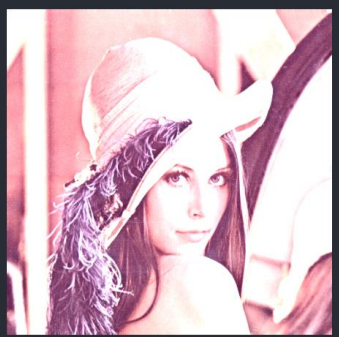
```
Citra GrayScale
```



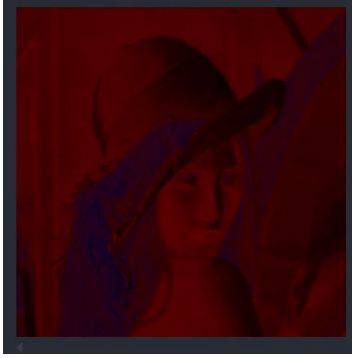
Program ini melakukan pembacaan gambar asli menggunakan library OpenCV, mengubahnya menjadi format Grayscale (hitam-putih) melalui fungsi cvtColor, menampilkan hasilnya dengan cv2_imshow, dan menyimpan hasil konversi tersebut ke dalam file baru bernama gambar.png.

```python
#Mebaca dua Citra
image1 = cv2.imread('gambar.jpg')
image2 = cv2.imread('gambar.png')

# Penambahan Citra
added_image = cv2.add(image1, image2)

# Pengurangan Citra
subtracted_image = cv2.subtract(image1, image2)

# Menampilkan Citra hasil penambahan dan pengurangan
print('Citra Hasil Penambahan')
cv2_imshow(added_image)
print('Citra Hasil Pengurangan')
cv2_imshow(subtracted_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
Citra Hasil Penambahan
```



```
Citra Hasil Pengurangan
```



Program ini melakukan pembacaan gambar asli menggunakan library **OpenCV**, mengubahnya menjadi format **Grayscale** (hitam-putih) melalui fungsi cvtColor, menampilkan hasilnya dengan cv2_imshow, dan menyimpan hasil konversi tersebut ke dalam file baru bernama gambar.png.

```python
# Perkalian Citra dengan Skalar
multiplied_image = cv2.multiply(image, 1.5)

# Menampikan Citra hasil Perkalian
print('Citra Hasil Perkalian')
cv2_imshow(multiplied_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
Citra Hasil Perkalian
```

Melakukan perkalian ada citra

```
# Menyesuaikan Kecerahan dan kontras
alpha = 3.5  # Faktor kecerahan
beta = 10 # nilai kontras
adjusted_image =cv2.convertScaleAbs(image, alpha=alpha, beta=beta)

# Menampilkan Citra yang sudah disesuaikan
print('Citra Hasil Penyesuaian')
cv2_imshow(adjusted_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
Citra Hasil Penyesuaian
```

Penyesuaian kontras

```python
# Aplikasikan Filter haussian
gambar_blur = cv2.GaussianBlur(image, (5, 5), 10)

# Tampilkan gambar asli dan hasil pengaburan
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.title('Citra Asli')
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title('Pengaburan dengan filter Gaussian')
plt.imshow(cv2.cvtColor(gambar_blur, cv2.COLOR_BGR2RGB))
plt.axis('off')
```

```
(np.float64(-0.5), np.float64(511.5), np.float64(511.5), np.float64(-0.5))
```

Filter gausian

```python
# Aplikasikan Filter Laplician
laplacian = cv2.Laplacian(image, cv2.CV_64F)

# Tampilkan gambar asli dan hasil pengasahan
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title('Citra Asli')
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title('Hasil Pengasahan dengan Filter Laplician')
plt.imshow(laplacian, cmap='gray')
plt.axis('off')

plt.show()
```

Python

```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or
```

Filter lapican

```python
# Aplikasikan Canny edge detection
cv2.Canny =cv2.Canny(image, 100, 200)

# Tampilkan gambar asli dan hasil Canny edge detection
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title('Citra Asli')
plt.imshow(img, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title('Hasil Canny edge detection')
plt.imshow(cv2.Canny, cmap='gray')
plt.axis('off')

plt.show()
```

Python

Edge detection

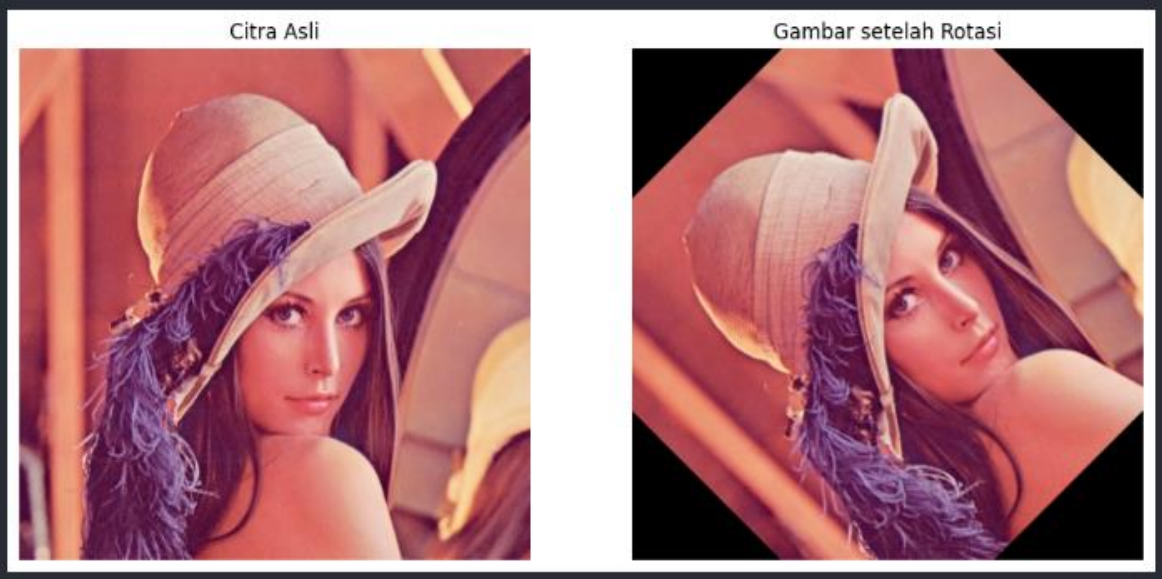
```python
# Ambil ukuran gambar
tinggi, lebar = img.shape[:2]

# Tentukan faktor scaling untuk x dan y
scaling_x = 0.1 # Scaling sebesar 50%
scaling_y = 0.1

# Tentukan matriks transformasi untuk scaling
M_scaling = np.float32([[scaling_x, 0, 0],
                        [0, scaling_y, 0]])

# Lakukan Scaling gambar
gambar_scaling = cv2.warpAffine(img, M_scaling, (int(lebar * scaling_x), int(tinggi * scaling_y)))

# Tampilkan gambar asli dan hasil scaling
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title('Citra Asli')
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title('Gambar setelah Scaling')
plt.imshow(cv2.cvtColor(gambar_scaling, cv2.COLOR_BGR2RGB))
plt.axis('off')
```

Python

```
(np.float64(-0.5), np.float64(50.5), np.float64(50.5), np.float64(-0.5))
```

Scaling

```python
# Ambil ukuran gambar
tinggi, lebar = img.shape[:2]

#Tentukan sudut rotasi (dalam derajat)
sudut_rotasi = 45

# Tentukan titik rotasi (di tengah gambar)
titik_rotasi = (lebar // 2, tinggi // 2)

# Tentukan matriks Transformasi untuk rotasi
M_rotasi = cv2.getRotationMatrix2D(titik_rotasi, sudut_rotasi, 1)

#Lakukan rotasi pada gambar
gambar_rotasi = cv2.warpAffine(img, M_rotasi, (lebar, tinggi))

# Tampilkan gambar asli dan hasil rotasi
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title('Citra Asli')
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title('Gambar setelah Rotasi')
plt.imshow(cv2.cvtColor(gambar_rotasi, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.show()
```



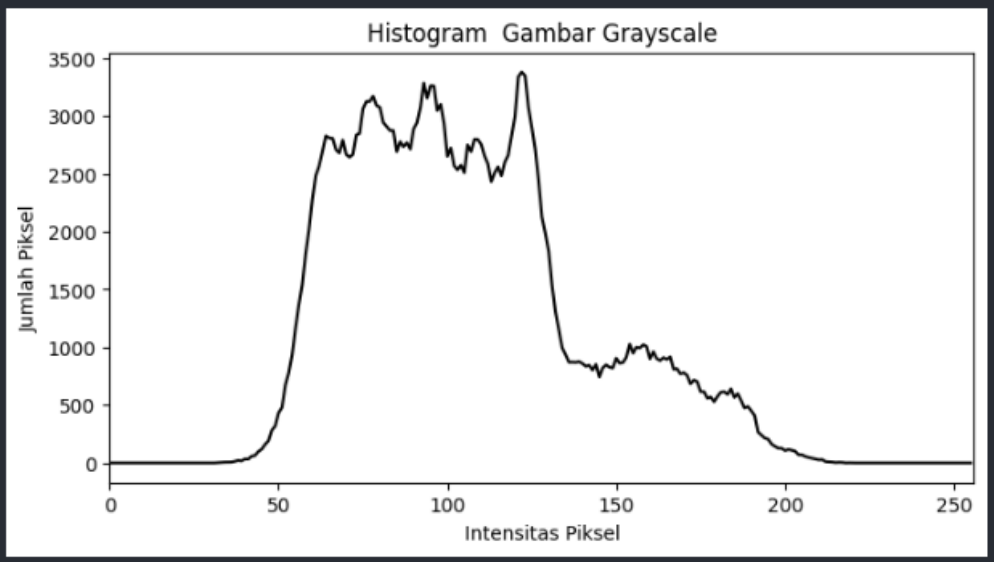
Rotasi

```python
# Ambil ukuran gambar
tinggi, lebar = img.shape[:2]

# Tentukan Matriks transalasi
# geser gambar sejauh 50 piksel ke kanan dan 30 piksel ke bawah
M = np.float32([[1, 0, 50],
                [0, 1, 30]])
# Lakukan translasi pada gambar
gambar_translasi = cv2.warpAffine(img, M, (lebar, tinggi))

# Tampilkan gambar asli dan Translasi
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title('Citra Asli')
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title('Gambar Setelah Translasi')
plt.imshow(cv2.cvtColor(gambar_translasi, cv2.COLOR_BGR2RGB))
plt.axis('off')

plt.show()
```

Python

Translasi

```python
# Menghitung Histogram
histogram = cv2.calcHist([img], [0], None, [256], [0, 256])

#Tampilkan Histogram
plt.figure(figsize=(8, 4))
plt.title('Histogram  Gambar Grayscale')
plt.xlabel('Intensitas Piksel')
plt.ylabel('Jumlah Piksel')
plt.plot(histogram, color='black')
plt.xlim([0, 256])
plt.show()
```



Histogram grayscale

--- Selamat Mengerjakan ---